



Nhóm 1 Thiết kế và điều khiển cánh tay robot bằng adruino

Tiếng anh ôn đầu ra ENG ESTE (Trường Đại Học Duy Tân)



messages.pdf_cover_qr_code_label

ỦY BAN NHÂN DÂN TP. HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT



Báo cáo môn học: CHUYÊN ĐỀ INTERNET OF THINGS (IoTs)

Tên đề tài

**Thiết kế và điều khiển cánh tay Robot gấp sản phẩm bằng
Arduino**

SVTH: TRẦN THANH ĐỒNG - MSSV: 2395202010006

HUỲNH NGUYỄN HOÀNG TRUNG - MSSV: 2395202010024

GVHD: ThS. NGÔ THANH ĐÔNG

TP.HCM, tháng 09 năm 2025

MỤC LỤC

MỤC LỤC	2
DANH MỤC CÁC HÌNH	4
DANH MỤC CÁC BẢNG.....	5
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	6
1.1. 1. Lý do chọn đề tài	6
1.2. 2. Mục tiêu của đề tài	6
1.3. 3. Ý nghĩa khoa học và thực tiễn.....	7
Chương 2. TỔNG QUAN ĐỀ TÀI	8
2.1. Tổng quan đề tài	8
2.1.1. Cơ sở lý thuyết về robot và cánh tay robot	8
2.1.2. Tình hình nghiên cứu và ứng dụng.....	8
2.1.3. Ưu điểm – hạn chế của cánh tay robot Arduino	8
2.2. Nội dung thực hiện	9
2.2.1. Nghiên cứu tài liệu và lý thuyết	9
Khái niệm về robot, cánh tay robot	9
2.2.2. Nguyên lý hoạt động của động cơ servo, Arduino	10
2.2.3. Phương pháp điều khiển cơ bản (thủ công, tự động)	10
2.2.4. Phân tích và lựa chọn giải pháp thiết kế.....	11
2.2.5. Thiết kế mạch điều khiển	13
2.2.6. Lập trình ESP 32.....	13
2.2.7. Thiết kế cơ khí.....	13
2.2.8. Thử nghiệm và hiệu chỉnh.....	16
2.2.9. Đánh giá kết quả và hướng phát triển	16
Chương 3. THIẾT KẾ VÀ THI CÔNG.....	17
3.1.1. Thiết kế hệ thống	17
3.1.2. Thiết kế cơ khí	17

3.1.3. Thiết kế điện – điều khiển	18
3.1.4. Thi công mô hình.....	19
Chương 4. KẾT QUẢ THỰC HIỆN.....	21
4.1. Tạo Firebase Project.....	21
4.2. Chương trình ESP32 nút nhấn điều khiển LED	28
4.3. Chương trình gửi dữ liệu từ ESP32 lên Firebase	30
Chương 5. KẾT LUẬN	43
5.1.1. Kết luận	43
5.1.2. Hướng phát triển.....	43
TÀI LIỆU THAM KHẢO.....	45

DANH MỤC CÁC HÌNH

Hình 1 Kết nối servo với arduino	10
Hình 2 Các khớp cánh tay robot.....	11
Hình 3 Động cơ DC giảm tốc JGB37-520	12
Hình 4 Động Cơ DC Servo JGB37-520	12
Hình 5 Mô phỏng.....	16
Hình 6 Điều khiển các khớp	18
Hình 7 Mô hình thử nghiệm	20
Hình 8 Firebase Project	21
Hình 9 Chọn trợ lý trong Firebase Project	22
Hình 10 Tạo dự án (Create project).....	23
Hình 11 Quá trình khởi tạo dự án	24
Hình 12 Điều khiển (console).....	25
Hình 13 Create Database (Tạo cơ sở dữ liệu).	25
Hình 14 Thiết lập vị trí cho Realtime Database	26
Hình 15 Đường dẫn các note của dữ liệu	26
Hình 16 Firebase Console để dễ theo dõi.....	27
Hình 17 Firebase để tạo node	27
Hình 18 Bo mạch esp 32	28

DANH MỤC CÁC BẢNG

Bảng 1 Các cấu tạo linh kiện cơ khí..... 14

Bảng 2 Bo mạch module ESP32-WROOM-32 28

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1. 1. Lý do chọn đề tài

Trong thời đại công nghiệp 4.0, tự động hóa và robot ngày càng đóng vai trò quan trọng trong sản xuất và đời sống. Robot công nghiệp, đặc biệt là cánh tay robot gấp sản phẩm, được ứng dụng rộng rãi trong dây chuyền sản xuất, lắp ráp và phân loại sản phẩm nhằm:

- Tăng năng suất lao động.
- Giảm thiểu rủi ro cho con người trong môi trường nguy hiểm.
- Đảm bảo độ chính xác và ổn định trong thao tác lắp đi lắp lại.
- Trong sản xuất hiện đại, robot là công cụ quan trọng vì chúng làm việc nhanh, chính xác và không biết mệt mỏi.
- Đặc biệt, cánh tay robot gấp sản phẩm có thể thay thế con người trong việc lắp lại thao tác đơn giản, nhưng cần chính xác (ví dụ: gấp linh kiện, sắp xếp sản phẩm).
- Do đó, chọn thiết kế mô hình nhỏ gọn, giá rẻ, dễ làm dựa trên Arduino (nền tảng lập trình mở, dễ học, phổ biến).

Tuy nhiên, robot công nghiệp chuyên dụng thường có giá thành cao, khó tiếp cận đối với sinh viên và các mô hình nghiên cứu nhỏ. Vì vậy, thiết kế và chế tạo một cánh tay robot đơn giản, sử dụng nền tảng mở Arduino là một hướng đi phù hợp để vừa tiết kiệm chi phí, vừa tạo điều kiện cho sinh viên rèn luyện kỹ năng thiết kế – lập trình – điều khiển.

1.2. 2. Mục tiêu của đề tài

- Thiết kế và chế tạo mô hình cánh tay robot 3–4 bậc tự do có khả năng gấp, di chuyển và thả sản phẩm.

- Sử dụng Arduino làm bộ điều khiển trung tâm để xử lý và điều khiển các động cơ servo.
- Xây dựng chương trình điều khiển với 2 chế độ:
- Thủ công (joystick, nút nhấn).
- Tự động (chu trình gấp – di chuyển – thả sản phẩm được lập trình sẵn).
- Ứng dụng trong các bài thí nghiệm, mô phỏng quy trình sản xuất và học tập về robot.

1.3. 3. Ý nghĩa khoa học và thực tiễn

- Ý nghĩa khoa học: Đề tài giúp sinh viên làm quen với các khái niệm về robot học, điều khiển servo, kinematics cơ bản và lập trình nhúng, phát triển sang robot có AI, IoT, thị giác máy.
- Ý nghĩa thực tiễn:
- Mô hình mô phỏng một dây chuyền sản xuất tự động thu nhỏ.
- Tạo tiền đề cho việc phát triển nền tảng nghiên cứu nếu muốn phát triển sang robot có AI, IoT, thị giác máy.

Chương 2. TỔNG QUAN ĐỀ TÀI

2.1. Tổng quan đề tài

2.1.1. Cơ sở lý thuyết về robot và cánh tay robot

Robot công nghiệp: Là thiết bị tự động có khả năng lập trình, thực hiện nhiều nhiệm vụ khác nhau như hàn, lắp ráp, sơn, vận chuyển, gấp – thả sản phẩm.

Cánh tay robot (Robotic Arm): Mô phỏng cánh tay con người, gồm nhiều khớp (bậc tự do – DOF), cho phép thực hiện các chuyển động linh hoạt. Các bậc tự do càng nhiều thì robot càng phức tạp và có thể thực hiện nhiều thao tác hơn.

Nguyên lý điều khiển: Thường sử dụng động cơ điện (servo, step motor, DC motor). Arduino hoặc PLC là bộ điều khiển trung tâm, nhận lệnh từ người dùng (nút nhấn, joystick, máy tính) hoặc từ chương trình cài đặt sẵn để điều khiển chuyển động robot.

2.1.2. Tình hình nghiên cứu và ứng dụng

Trên thế giới: Các hãng lớn như ABB, KUKA, Yaskawa, FANUC... đã phát triển robot công nghiệp 6–7 bậc tự do có khả năng hàn, lắp ráp, sơn, bốc dỡ hàng hóa. Đây là các robot thương mại với độ chính xác cao, tốc độ nhanh nhưng giá thành rất đắt.

Ở Việt Nam: Một số trường đại học, viện nghiên cứu và doanh nghiệp đang chế tạo robot mô hình hoặc robot gấp sản phẩm phục vụ đào tạo. Tuy nhiên, đa phần là quy mô nhỏ, dùng cho giảng dạy và nghiên cứu, chưa phổ biến trong công nghiệp thực tế.

Xu hướng hiện nay: Ứng dụng nền tảng mở (Arduino, Raspberry Pi) kết hợp với công nghệ IoT, AI, thị giác máy để tạo ra các robot thông minh, chi phí thấp, phù hợp cho đào tạo và thí nghiệm.

2.1.3. Ưu điểm – hạn chế của cánh tay robot Arduino

Ưu điểm:

- Chi phí thấp, dễ chế tạo.
- Dễ học, dễ lập trình nhờ Arduino IDE.
- Nghiên cứu cơ bản.

Hạn chế:

- Độ chính xác và độ bền không cao.
- Phù hợp cho mô hình thí nghiệm

Lý do lựa chọn hướng nghiên cứu

- Xuất phát từ nhu cầu học tập và nghiên cứu của sinh viên ngành điện – điện tử – cơ điện tử.
- Tận dụng nền tảng Arduino phổ biến, giúp dễ dàng tiếp cận và mở rộng.
- Mô hình có tính thực tiễn cao, phục vụ thí nghiệm, mô phỏng sản xuất tự động.
- Robot gấp sản phẩm là hướng nghiên cứu quan trọng và có ứng dụng thực tế rộng rãi.
- Arduino là nền tảng phù hợp để chế tạo mô hình robot giá rẻ, dễ tiếp cận .

2.2. Nội dung thực hiện

2.2.1. Nghiên cứu tài liệu và lý thuyết

Khái niệm về robot, cánh tay robot

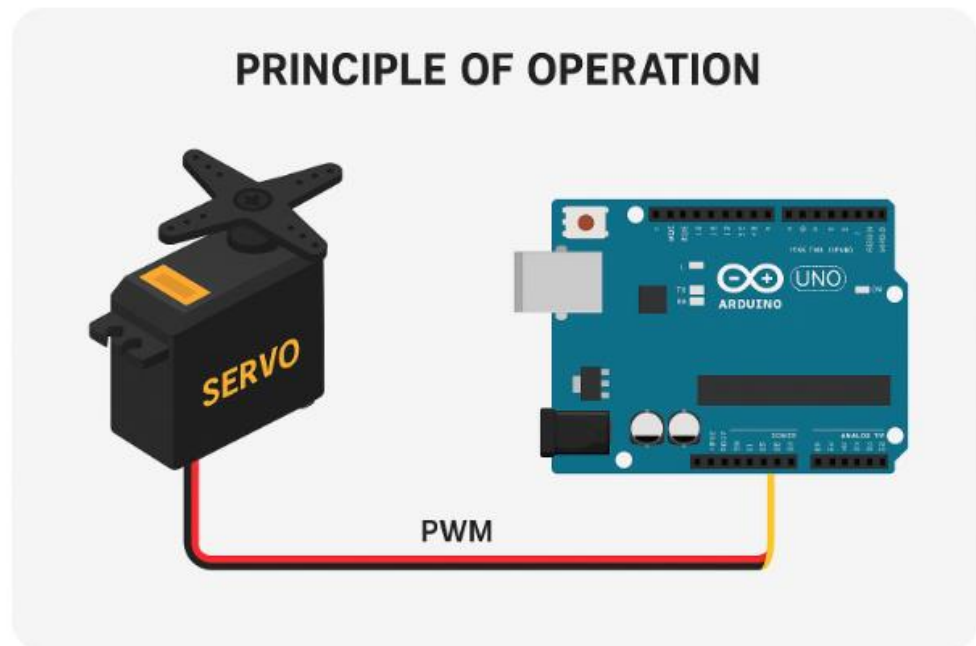
Robot là thiết bị tự động, có khả năng lập trình để thực hiện các thao tác lặp đi lặp lại thay cho con người.

Cánh tay robot là một dạng robot công nghiệp mô phỏng cấu trúc và chuyển động của tay người, có nhiều bậc tự do (DOF), giúp thực hiện các công việc như gấp, di chuyển, lắp ráp sản phẩm.

Ứng dụng phổ biến: trong dây chuyền sản xuất ô tô, điện tử, bao gói, phân loại sản phẩm, và các công việc nguy hiểm như hàn, sơn, làm việc trong môi trường độc hại.

2.2.2. Nguyên lý hoạt động của động cơ servo, Arduino

- Động cơ servo: là động cơ có khả năng quay một góc xác định (0° – 180° hoặc 0° – 360°) nhờ tín hiệu điều khiển xung PWM. Thường dùng để điều khiển khớp quay trong robot.
- Arduino: nền tảng vi điều khiển mã nguồn mở, dễ lập trình bằng Arduino IDE. Arduino có các chân PWM để xuất tín hiệu điều khiển servo, đồng thời có thể nhận tín hiệu từ joystick, nút nhấn.
- Arduino được chọn vì chi phí thấp, dễ lập trình, cộng đồng hỗ trợ rộng rãi.



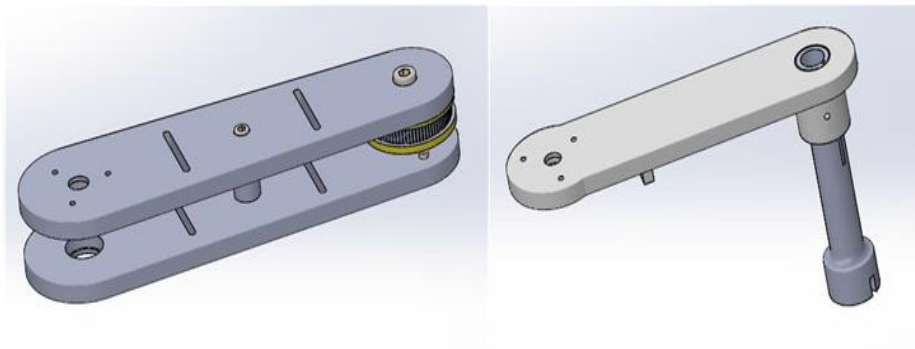
Hình 1 Kết nối servo với arduino

2.2.3. Phương pháp điều khiển cơ bản (thủ công, tự động)

- Điều khiển thủ công: trực tiếp điều khiển robot thông qua joystick hoặc nút nhấn → Arduino đọc tín hiệu và điều chỉnh góc quay servo theo thao tác.
- Điều khiển tự động: robot thực hiện các chu trình gấp – di chuyển – thả sản phẩm được lập trình sẵn trong Arduino.

2.2.4. Phân tích và lựa chọn giải pháp thiết kế

- Số bậc tự do của robot (3–4 DOF)
- Số khớp chuyển động độc lập của robot.
- Mục tiêu mô phỏng thao tác gấp – di chuyển – thả sản phẩm, robot cần tối thiểu 3 bậc tự do:
- Để xoay (quay trái – phải).
- Khớp nâng/hạ (điều chỉnh độ cao).
- Khớp gấp (mở – đóng kẹp).



Hình 2 Các khớp cánh tay robot

Chọn loại động cơ servo phù hợp:

- Mô-men xoắn đủ để nâng cánh tay và vật gấp.
- Kích thước nhỏ gọn, phù hợp với mô hình.
- Góc quay phù hợp (0° – 180° cho khớp, 0° – 360° cho để xoay).

Đề xuất:

Động cơ DC giảm tốc JGB37-520 : dùng cho khớp gấp (nhỏ, nhẹ).

	- Kích thước mặt bích: 42×42mm. - Dòng chịu tải: 1.6A. - Momen xoắn: 0.45Nm. - Góc bước: 1.8°/step. - Đường kính trục 5mm.
---	--

Hình 3 Động cơ DC giảm tốc JGB37-520

Servo Động Cơ DC Servo JGB37-520 dùng cho khớp nâng/hạ, để xoay (mô-men xoắn lớn hơn).

	- Điện áp hoạt động: Từ 6~15V DC. - Điện áp định mức: 12V DC. - Dòng điện không tải ở 12V: 60 mA. - Mô-men xoắn ở 12V: 5 kg.cm. - Tỷ lệ bánh răng: 1:90.
---	--

Hình 4 Động Cơ DC Servo JGB37-520

Lựa chọn vật liệu chế tạo khung

Mica: dễ gia công, giá rẻ, trọng lượng nhẹ.

Nhựa in 3D (PLA, ABS): cho phép chế tạo chi tiết chính xác, hình dạng linh hoạt.

Kim loại mỏng (nhôm, thép nhẹ): bền chắc hơn, phù hợp làm đế hoặc trục chính.

Kết hợp: đế bằng kim loại mỏng để vững chắc, các bộ phận tay bằng mica hoặc nhựa in 3D để giảm trọng lượng.

Đề xuất sơ đồ khối hệ thống điều khiển

Arduino (bộ điều khiển trung tâm): xử lý tín hiệu và phát xung PWM cho servo.

Joystick / Nút nhấn: thiết bị nhập lệnh điều khiển thủ công.

Servo: cơ cấu chấp hành, điều khiển các khớp robot.

Nguồn điện DC: cấp nguồn cho servo và Arduino.

2.2.5. Thiết kế mạch điều khiển

Lập sơ đồ nguyên lý kết nối: Esp 32, servo, joystick, nút nhấn, nguồn.

Lắp đặt mạch điều khiển trên breadboard hoặc PCB nhỏ gọn.

Kiểm tra khả năng cấp nguồn và tín hiệu điều khiển PWM.

2.2.6. Lập trình ESP 32

- Viết chương trình điều khiển chế độ thủ công: đọc tín hiệu joystick, nút nhấn → điều khiển góc quay servo.
- Viết chương trình điều khiển chế độ tự động: lập trình sẵn chuỗi thao tác gấp – di chuyển – thả.
- Tối ưu chương trình để robot hoạt động mượt và ổn định.

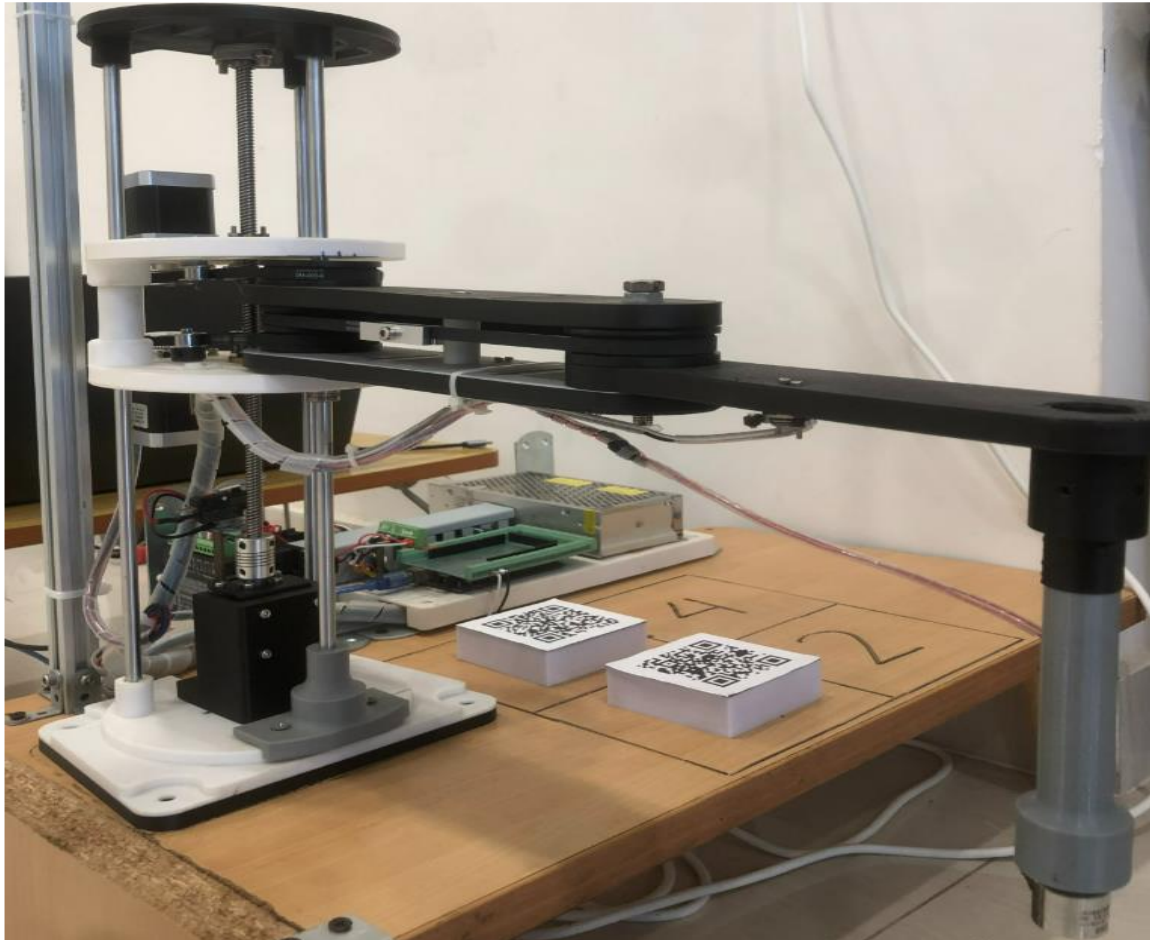
2.2.7. Thiết kế cơ khí

Bảng 1 Các cấu tạo linh kiện cơ khí

Số thứ tự	Tên	Hình ảnh	Thông số
1	Puly 20 răng trục 5mm		- Chiều dài: 16mm. - Cân nặng: 5g. - Đường kính đầu vào: 5mm. - Đường kính đầu ra: 5mm. - Số răng: 20 răng.
2	Puly đôi 80 răng trục 5mm		- Số răng: 80 răng. - Cân nặng: 10g. - Bán rộng: 6mm. - Đường kính đầu vào: 5mm. - Đường kính đầu ra: 5mm.
3	Ròng rọc trơn trục 5mm		- Loại: ròng rọc trơn. - Bề rộng rãnh đai: 6.5mm. - Đường kính lỗ trục: 5mm.
4	Gối đỡ vòng bi bạc đạn KLF06		- Chất liệu: hợp kim. - Khối lượng: 30g. - Trục vòng bi: 8mm.

5	Con trượt tròn LM8UU		- Chất liệu: kim loại. - Màu sắc: màu kim loại. - Đường kính: 15mm. - Chiều dài: 24mm. - Dành cho trục: 8mm. - Khối lượng: 13.1g.
6	Puly GT2 80 răng		- Số răng: 80 răng. - Bề rộng đai: 6mm. - Đường kính ngoài: 54mm. - Đường kính bánh răng: 50.5mm. - Bước ren: 2mm. - Chiều cao: 20mm. - Chất liệu: nhôm.
7	Dây đai GT2 300		- Bước răng: 2mm. - Chiều rộng đai: 6mm.
8	Bạc đạn 6087		- Đường kính trong: 10 mm. - Đường kính ngoài: 26mm. - Độ dày: 8mm. - Khối lượng: 0.019 kg.
9	Mặt bích trục 8mm		- Vật liệu: Thép. - Kích thước lỗ trục bên trong: 8mm. - Đường kính ngoài: 16mm, 32mm. - Tổng chiều cao: 13mm. - Kích thước vít: M4.

2.2.8. Thử nghiệm và hiệu chỉnh



Hình 5 Mô phỏng

- Thử nghiệm robot ở chế độ thủ công và tự động.
- Quan sát hoạt động: tốc độ, độ chính xác, độ ổn định.
- Điều chỉnh phần mềm và cơ khí nếu phát hiện lỗi.

2.2.9. Đánh giá kết quả và hướng phát triển

- Đánh giá ưu điểm: chi phí thấp, dễ chế tạo.
- Chỉ ra hạn chế: tải trọng thấp, độ chính xác chưa cao, chưa có AI
- Phát triển: thêm bậc tự do, tích hợp cảm biến, IoT, thị giác máy tính.

Chương 3. THIẾT KẾ VÀ THI CÔNG

3.1.1. Thiết kế hệ thống

Mục tiêu thiết kế: xây dựng một cánh tay robot có khả năng xoay để, nâng hạ cánh tay và gấp sản phẩm, điều khiển thông qua Arduino và các thiết bị ngoại vi (joystick, nút nhấn).

Sơ đồ khối tổng thể:

Nguồn cấp điện → Arduino Uno → Servo Động Cơ DC Servo JGB37-520 → Cơ cấu cơ khí (để, khớp, kẹp gấp).

Thiết bị điều khiển: Joystick/Nút nhấn → Arduino.

Sơ đồ khối minh họa:

[Nguồn 5V/12V] → [Arduino Uno] → [Servo 1 (Để xoay)]
→ [Servo 2 (Khớp nâng/hạ)]
→ [Servo 3 (Kẹp gấp)]

[Joystick / Nút nhấn] → [Arduino Uno]

3.1.2. Thiết kế cơ khí

Cấu trúc chính:

Để xoay: giữ ổn định robot, cho phép xoay 180° – 270° .

Khớp nâng/hạ: thay đổi chiều cao tay gấp.

Khớp gấp: cơ cấu kẹp mở/đóng để gấp vật nhỏ.

Vật liệu: mica trong suốt 3–5mm, nhựa in 3D, hoặc kim loại mỏng (tùy khả năng gia công).

3.1.3. Thiết kế điện – điều khiển

Bộ điều khiển trung tâm: Arduino Uno R3.

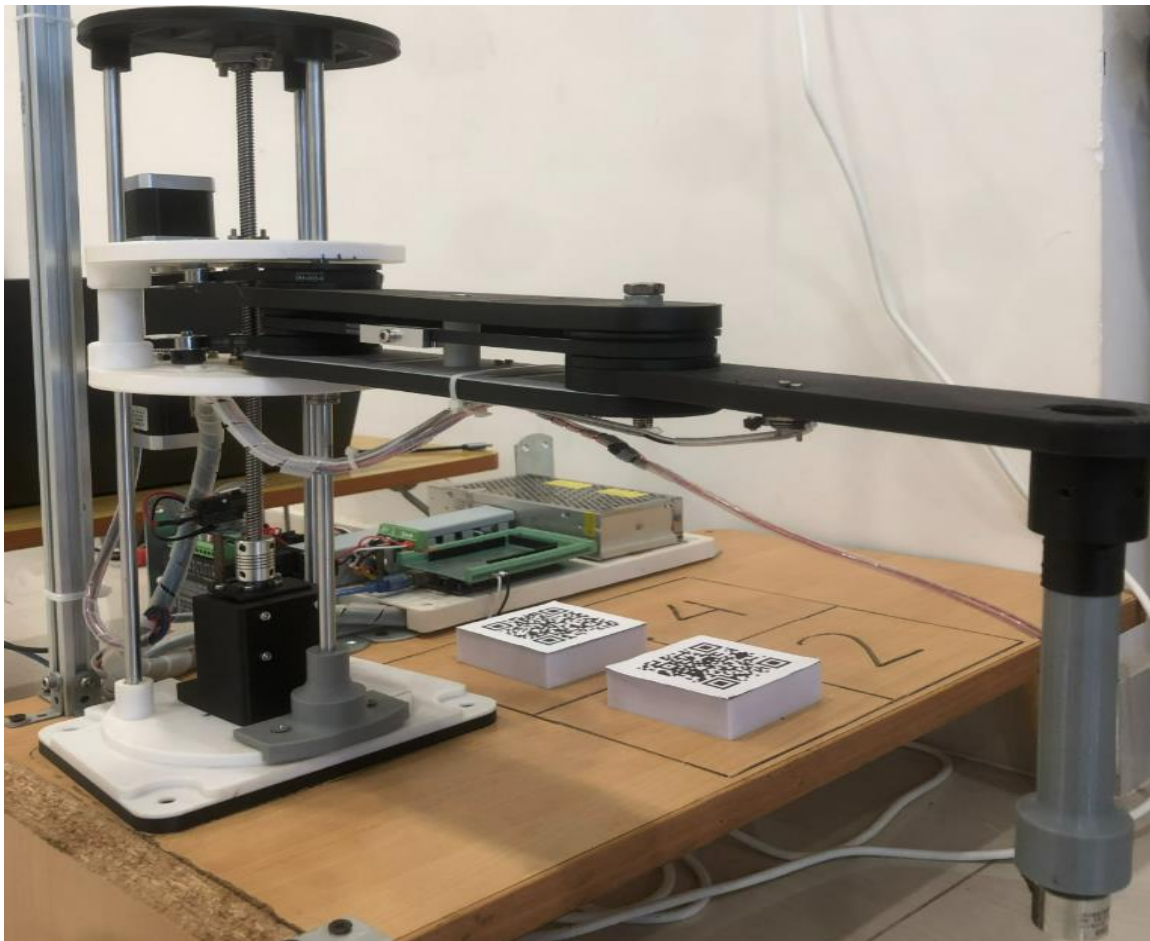
Cơ cấu chấp hành:

Động cơ DC giảm tốc JGB37-520 (2kg.cm) cho khớp chính (nâng/hạ).

Servo Động Cơ DC Servo JGB37-520 cho các khớp nhẹ (xoay đế, kẹp gấp).

Thiết bị điều khiển: Joystick module + nút nhấn để điều khiển thao tác thủ công.

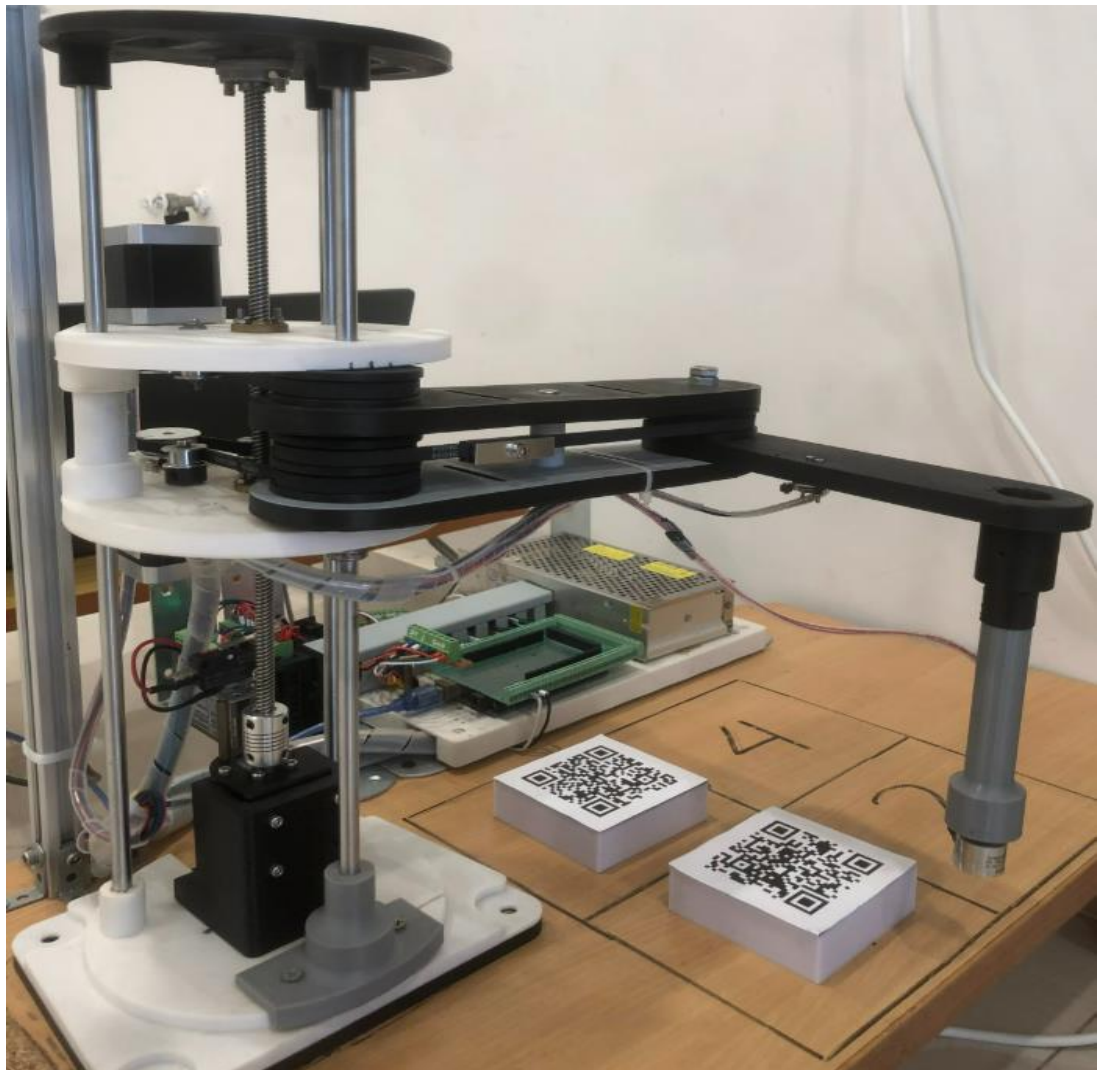
Nguồn cấp: 5V–12V, dòng tối thiểu 2A (tùy số lượng servo).



Hình 6 Điều khiển các khớp

3.1.4. Thi công mô hình

- Gia công – lắp ráp cơ khí:
- Cắt/gia công mica theo thiết kế CAD.
- Gắn các servo vào khung theo đúng vị trí thiết kế.
- Lắp ráp các khớp nối với vít và bu-lông nhỏ.
- Lắp ráp mạch điện:
- Kết nối Arduino với các servo (chân PWM).
- Kết nối joystick và nút nhấn vào Arduino (chân analog/digital).
- Kiểm tra nguồn cấp ổn định.
- Lập trình và chạy thử nghiệm:
- Lập trình Arduino bằng IDE để điều khiển servo theo tín hiệu joystick/nút nhấn.
- Kiểm tra các khớp di chuyển đúng hướng, điều chỉnh giới hạn góc xoay để tránh va chạm.
- Thử nghiệm gấp vật có khối lượng nhỏ (bóng nhựa, khối gỗ nhẹ).

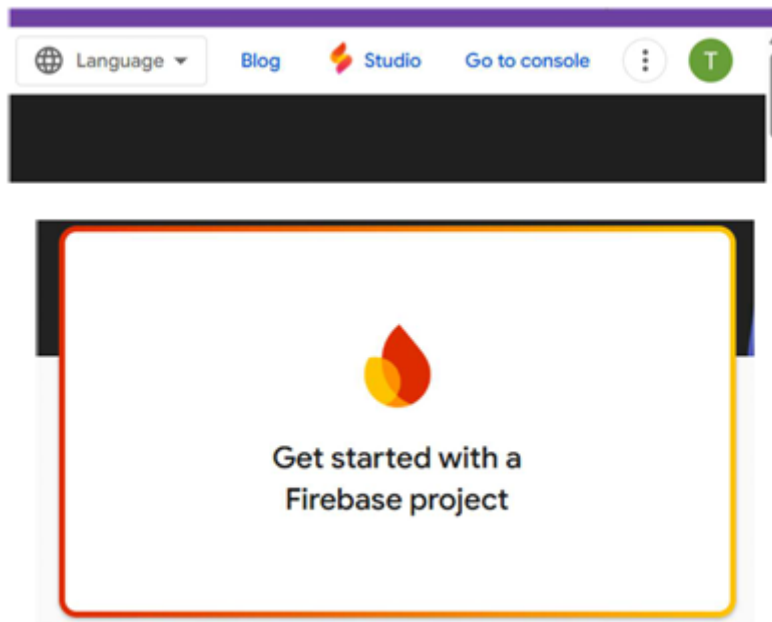


Hình 7 Mô hình thử nghiệm

Chương 4. KẾT QUẢ THỰC HIỆN

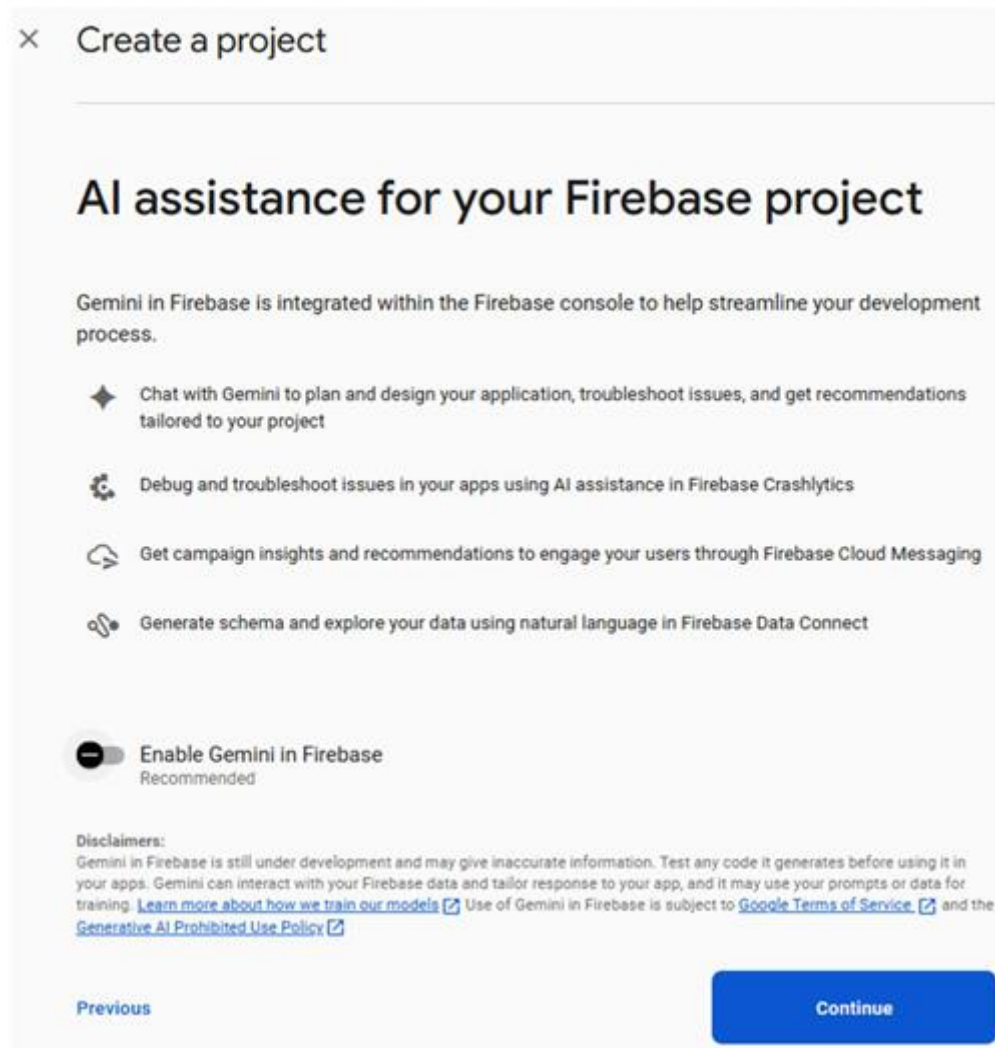
4.1. Tạo Firebase Project

- Truy cập Firebase và đăng nhập bằng tài khoản Google.
- Vào Firebase Console và tạo một dự án mới.



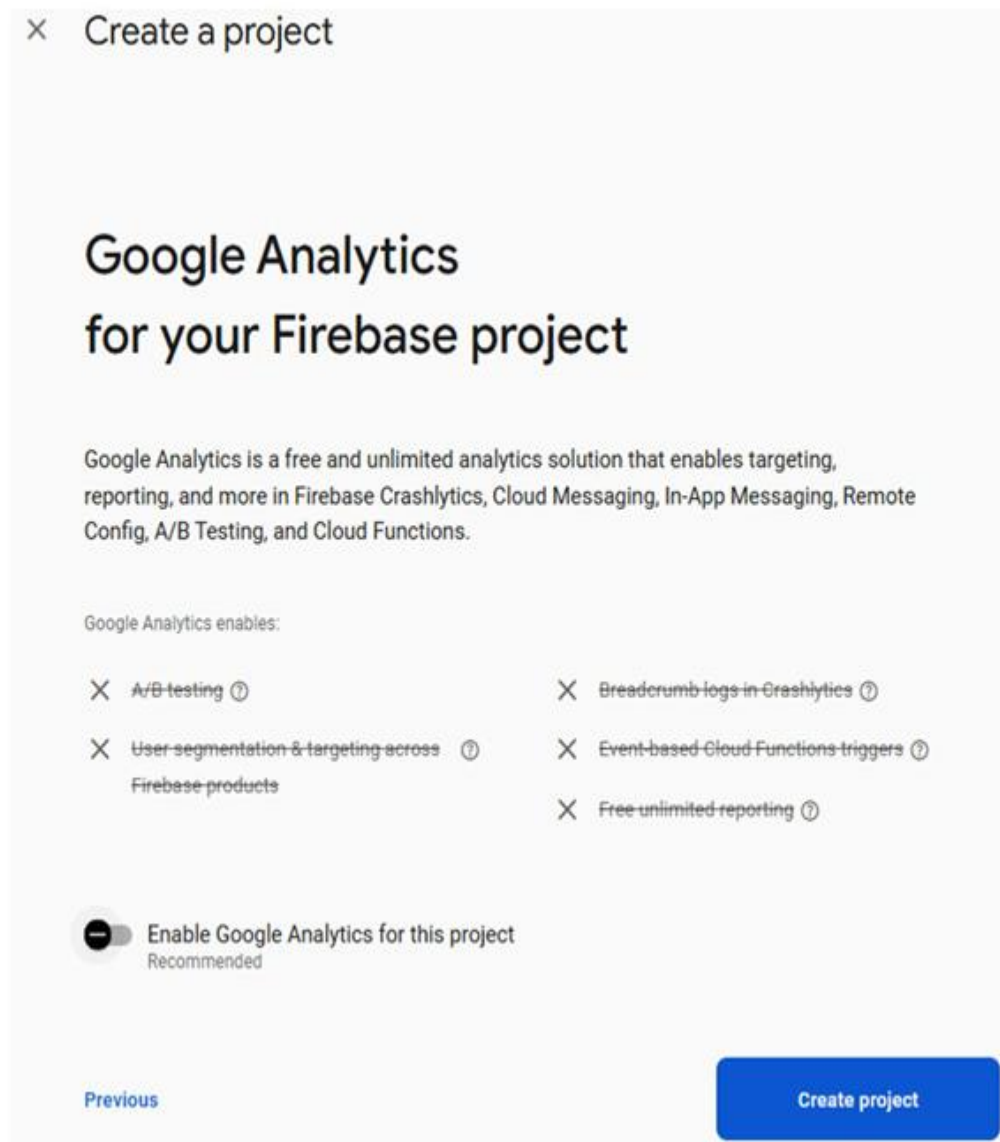
Hình 8 Firebase Project

- Đặt tên cho dự án, ví dụ: ESP32-IoTs-TDMU, sau đó nhấn Continue.
- Tiếp theo, bật hoặc tắt trợ lý AI (AI assistance) cho dự án của bạn. Tùy chọn này không bắt buộc. Sau đó nhấn Continue.



Hình 9 Chọn trợ lý trong Firebase Project

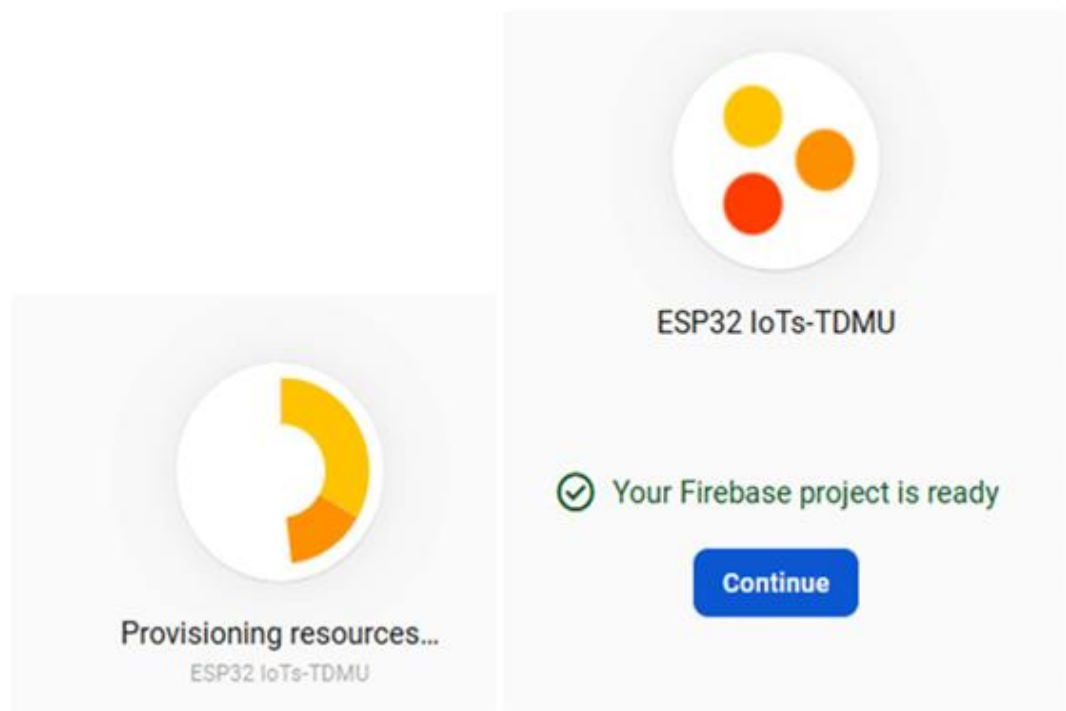
- Tắt tùy chọn "Enable Google Analytics for this project" vì dự án này không cần đến phân tích dữ liệu. Sau đó nhấn Tạo dự án (Create project).



Hình 10 Tạo dự án (Create project)

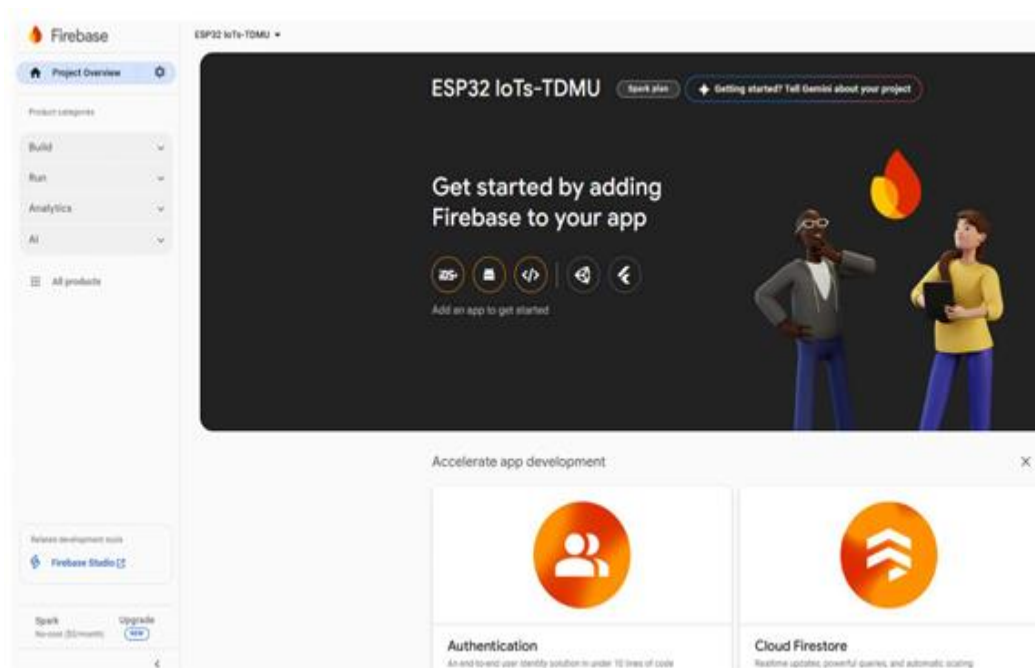
- Quá trình khởi tạo dự án sẽ mất vài giây. Khi hoàn tất, nhấn Continue.

Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Aduino



Hình 11 Quá trình khởi tạo dự án

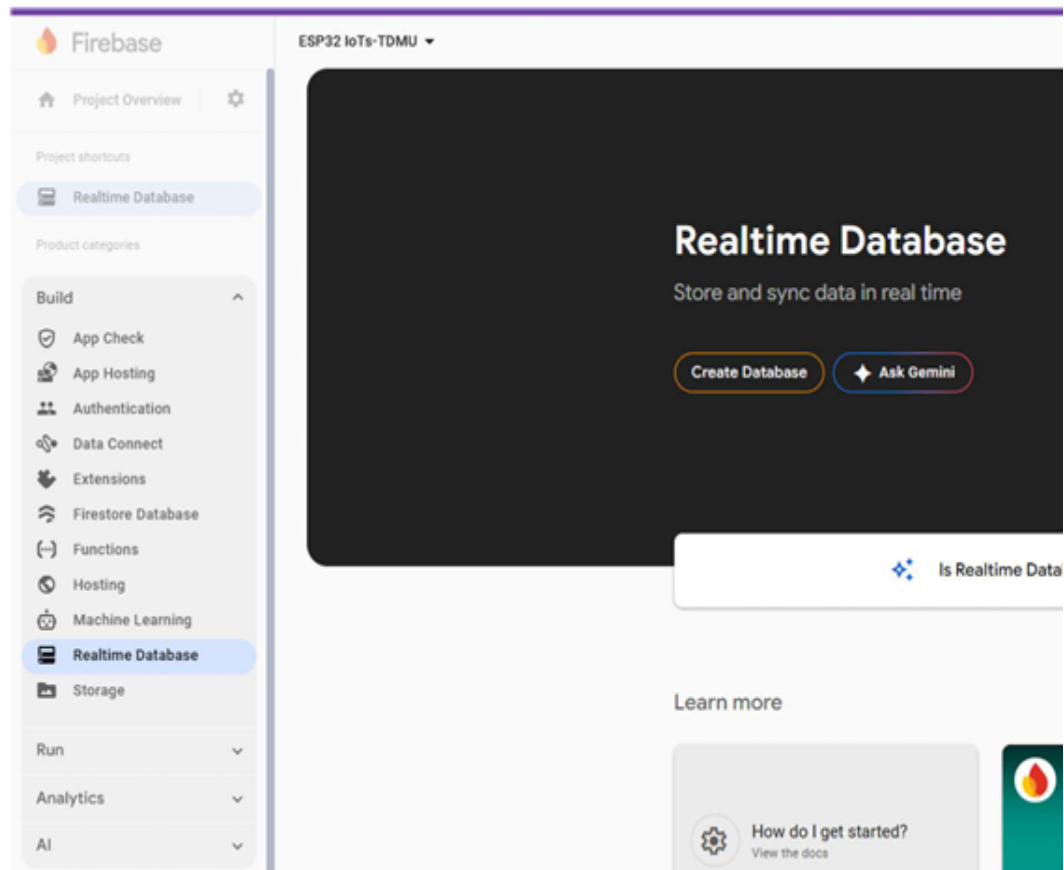
- Chuyển đến trang điều khiển (console) của dự án.



Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Aduino

Hình 12 Điều khiển (console)

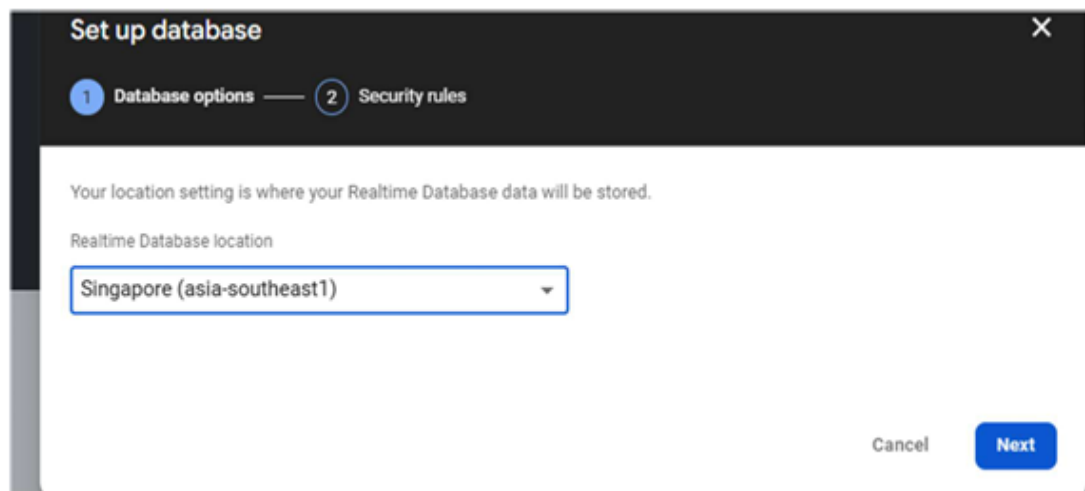
- Ở thanh bên trái, nhấp vào Realtime Database, sau đó nhấp vào Create Database (Tạo cơ sở dữ liệu).



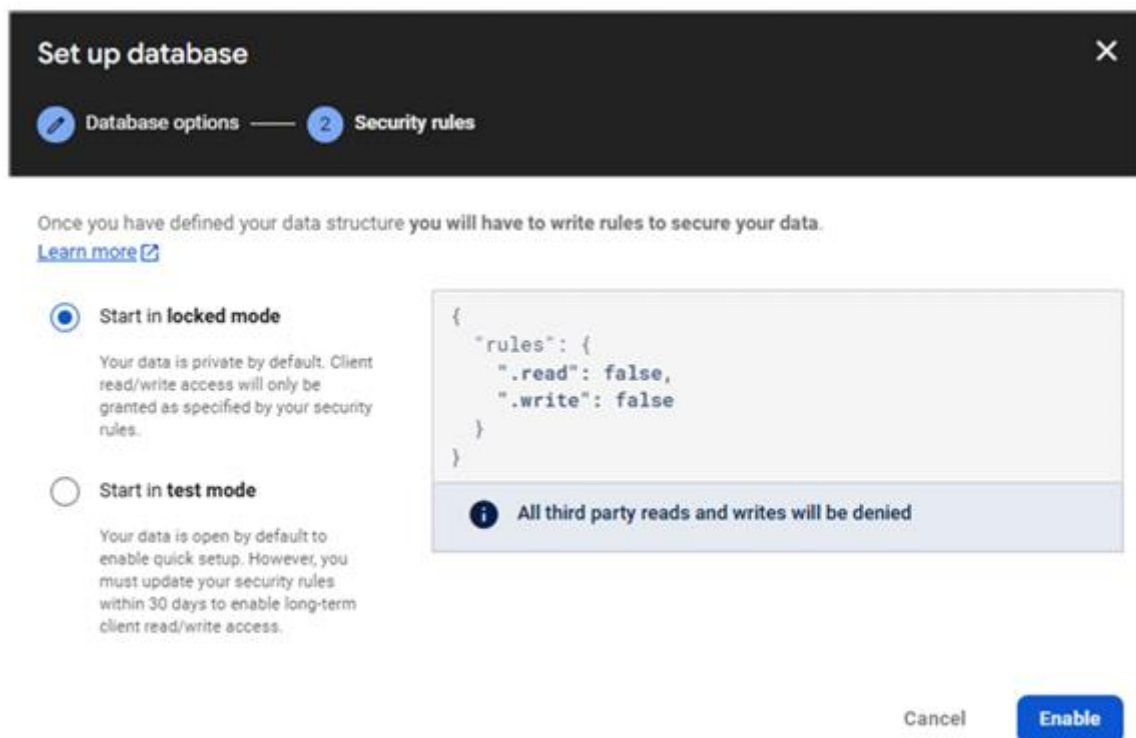
Hình 13 Create Database (Tạo cơ sở dữ liệu).

- Chọn vị trí địa lý cho cơ sở dữ liệu của bạn. Nên chọn vị trí gần nơi bạn đang sinh sống nhất để có tốc độ truy cập tốt hơn.

Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Arduino



Hình 14 Thiết lập vị trí cho Realtime Database

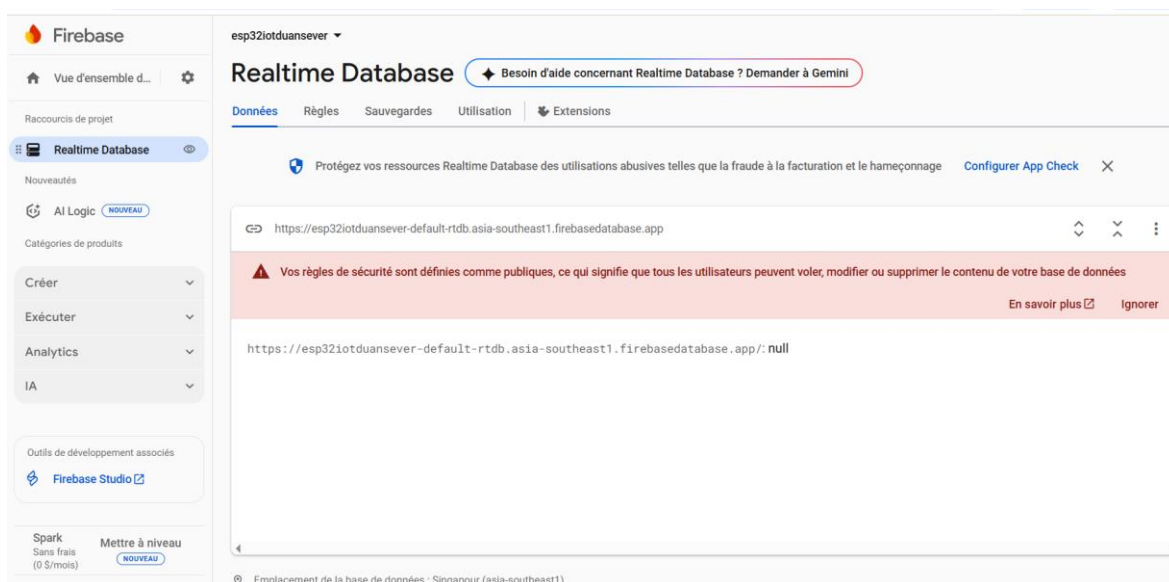


Hình 15 Đường dẫn các note của dữ liệu

Thẻ Data là đường dẫn các note của dữ liệu

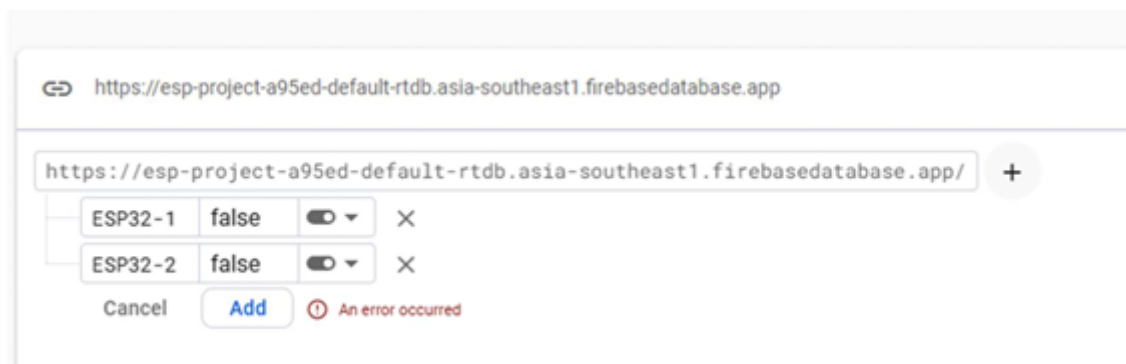
Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Aduino

Bây giờ, chúng ta sẽ tạo các node cơ sở dữ liệu. Bạn có thể tạo thủ công trực tiếp trên Firebase Console, qua ứng dụng web, hoặc từ ESP32. Trong hướng dẫn này, ta sẽ tạo thủ công trên Firebase Console để dễ theo dõi.



Hình 16 Firebase Console để dễ theo dõi

Bước 2: Bạn có thể dùng biểu tượng dấu (+) trong Firebase để tạo node. Tuy nhiên, để tránh gõ sai, bạn có thể tải một tệp JSON có sẵn mà chúng tôi cung cấp để tạo các node giống hướng dẫn.



Hình 17 Firebase để tạo node

Bước 3: Quay lại Firebase Console > Realtime Database. Nhấn vào biểu tượng ba dấu chấm ở góc phải, chọn Import JSON.

Bước 4: Chọn tệp .json bạn vừa tải xuống.

Bước 5: Sau khi import, cơ sở dữ liệu của bạn sẽ có cấu trúc như hình dưới (giống như đoạn JSON ở trên).

4.2. Chương trình ESP32 nút nhấn điều khiển LED



Hình 18 Bo mạch esp 32

Bảng 2 Bo mạch module ESP32-WROOM-32

Pin	Pin Label	GPIO	Reason
4	SENSOR_VP	GPIO36	Input only GPIO, cannot be configured as output
5	SENSOR_VN	GPIO39	Input only GPIO, cannot be configured as output
6	IO34	GPIO34	Input only GPIO, cannot be

			configured as output
7	IO35	GPIO35	Input only GPIO, cannot be configured as output
8	IO32	GPIO32	
9	IO33	GPIO33	
10	IO25	GPIO25	
11	IO26	GPIO26	
12	IO27	GPIO27	
13	IO14	GPIO14	
14	IO12	GPIO12	must be LOW during boot
16	IO13	GPIO13	
17	SHD/SD2	GPIO9	Connected to Flash memory
18	SWP/SD3	GPIO10	Connected to Flash memory
19	SCS/CMD	GPIO11	Connected to Flash memory
20	SCK/CLK	GPIO6	Connected to Flash memory
21	SDO/SD0	GPIO7	Connected to Flash memory
22	SDI/SD1	GPIO8	Connected to Flash memory
23	IO15	GPIO15	must be HIGH during boot, prevents startup log if pulled LOW
24	IO2	GPIO2	must be LOW during boot and also connected to the on-board LED
25	IO0	GPIO0	must be HIGH during boot and LOW for programming
26	IO4	GPIO4	

27	IO16	GPIO16	
28	IO17	GPIO17	
29	IO5	GPIO5	must be HIGH during boot
30	IO18	GPIO18	
31	IO19	GPIO19	
33	IO21	GPIO21	
34	RXD0	GPIO3	Rx pin, used for flashing and debugging
35	TXD0	GPIO1	Tx pin, used for flashing and debugging
36	IO22	GPIO22	
37	IO23	GPIO23	

4.3. Chương trình gửi dữ liệu từ ESP32 lên Firebase

```
// Thư viện cho Web Server và WiFi
#include <WiFi.h>
#include <WebServer.h>
#include <HTTPClient.h>

// Thư viện điều khiển động cơ từ chương trình 1
#include <AccelStepper.h>

// =====
// PHẦN 1: KHAI BÁO TỪ CHƯƠNG TRÌNH GỐC (CÁNH TAY ROBOT)
// Đã cập nhật với các chân ESP32 bạn cung cấp
// =====

// Khai báo chân cho ĐỘNG CƠ DC (TRỤC 1)
#define encoderPinA 27 // Chân A của encoder
#define encoderPinB 26 // Chân B của encoder
#define motorPin1 25 // IN1 trên DRV8833
#define motorPin2 33 // IN2 trên DRV8833
#define enB 32 // Chân PWM điều khiển tốc độ DC

// Khai báo chân cho ĐỘNG CƠ BƯỚC 1 (TRỤC 2)
#define stepper1_pin1 18 // Chân step của step1 (Lưu ý: Pin 34, 35 trên
ESP32 thường chỉ là input, đã đổi sang pin hợp lệ)
#define stepper1_pin2 5 // Chân dir của step1
```

Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Arduino

```
#define EnStep1 19          // Chân Enable của step1

// Khai báo chân cho ĐỘNG CƠ BƯỚC 2 (TRỤC 3)
#define stepper2_pin1 23    // Chân step của step2
#define stepper2_pin2 22    // Chân dir của step2
#define EnStep2 21          // Chân Enable của step2

// Khai báo đối tượng động cơ bước
AccelStepper stepper1(AccelStepper::DRIVER, stepper1_pin1, stepper1_pin2);
AccelStepper stepper2(AccelStepper::DRIVER, stepper2_pin1, stepper2_pin2);

// Biến cho encoder
volatile long int EncoderCount = 0;

// Hằng số cho chế độ điều khiển manual (jogging)
#define MANUAL_DC_SPEED 200 // Tốc độ PWM cho DC (0-255)
#define MANUAL_STEPPER_SPEED 800 // Tốc độ cho động cơ bước

// Hàm ngắt để cập nhật Encoder, port từ chương trình 1
void IRAM_ATTR updateEncoder() {
    // Logic đọc encoder đơn giản (không có bộ lọc như chương trình 1)
    if (digitalRead(encoderPinA) == digitalRead(encoderPinB)) {
        EncoderCount++;
    } else {
        EncoderCount--;
    }
}

// =====
// PHẦN 2: CHƯƠNG TRÌNH WEB SERVER (KHÔNG THAY ĐỔI GIAO DIỆN)
// =====

// Khai báo mạng wifi và pass
const char* ssid = "CHI CONG";
const char* password = "aA@162879";
// Khai báo biến class server
WebServer server(80);

// Khai báo thêm các chân LED (Giữ lại để tương thích với HTML/JS)
#define LED1 2 // Ví dụ dùng led onboard
#define LED2 15
#define LED3 4
```

```
// Biến lưu trạng thái
String SYSTEM_State = "OFF";
String LED1_State = "OFF"; // Trạng thái trục 1 (DC)
String LED2_State = "OFF"; // Trạng thái trục 2 (Step1)
String LED3_State = "OFF"; // Trạng thái trục 3 (Step2)

void handle_MainPage();

// Các hàm handle... sẽ được định nghĩa sau khi setup
void handleSystemOn();
void handleSystemOff();
void handleResetSystem();
void handleTruc1CW();
void handleTruc1CCW();
void handleTruc1Off();
void handleTruc2CW();
void handleTruc2CCW();
void handleTruc2Off();
void handleTruc3CW();
void handleTruc3CCW();
void handleTruc3Off();
void handleSystemState();

// --- HTML PAGE (GIỮ NGUYÊN) ---
const char* htmlPage = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>ESP32 Web Server</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 40px; }
    .circle { width:20px; height:20px; border-radius:50%; display:inline-
block; border:1px solid #aaa; margin-left:10px; }
    .circle.on, .circle.cw, .circle.ccw { background-color: green; }
    .circle.off { background-color: gray; }
    .button { border:none; color:white; padding:10px 25px; margin:10px;
font-size:16px; cursor:pointer; border-radius:8px; }
    .button.SYSTEMon { background-color: DeepSkyBlue; }
    .button.SYSTEMoff { background-color: Red; }
    .button.SYSTEM { background-color: DimGray; }
    .button.reset { background-color: Orange; }
    .status.enabled { color: green; font-weight: bold; }
```

```
.status.disabled { color: red; font-weight: bold; }
</style>
</head>
<body>
  <h2>Điều khiển cánh tay robot bằng ESP32</h2>
  <p>Điều khiển bằng tay (Jogging Mode)</p>

  <div style="margin: 20px 0;">
    <div style="display: flex; align-items: center; justify-content:
center; gap: 10px; margin-bottom: 10px;">
      <span>Trạng thái hệ thống: </span>
      <span id="sensorStatus" class="status enabled">KHÔNG HOẠT
ĐỘNG</span>
    </div>
    <div style="display: flex; align-items: center; justify-content:
center; gap: 10px;">
      <span id="SYSTEMtext_state">OFF</span>
      <div id="SYSTEMCircle_state" class="circle off"></div>
      <button id="button_SYSTEM_On" class="button SYSTEMon">HOẠT
ĐỘNG</button>
      <button id="button_SYSTEM_Off" class="button
SYSTEMoff">DỪNG</button>
      <button id="resetSYSTEM" class="button reset">RESET</button>
    </div>
  </div>

  <div style="display: flex; align-items: center; justify-content: center;
gap: 10px;">
    <span>Trục 1 (DC):</span>
    <div id="LED1Circle_state" class="circle off"></div>
    <button id="button_Truc1_CW" class="button SYSTEM">CW (Thuận)</button>
    <button id="button_Truc1_CCW" class="button SYSTEM">CCW
(Nghịch)</button>
    <button id="button_Truc1_OFF" class="button SYSTEM">DỪNG</button>
  </div>

  <div style="display: flex; align-items: center; justify-content: center;
gap: 10px;">
    <span>Trục 2 (Step1):</span>
    <div id="LED2Circle_state" class="circle off"></div>
    <button id="button_Truc2_CW" class="button SYSTEM">CW (Thuận)</button>
    <button id="button_Truc2_CCW" class="button SYSTEM">CCW
(Nghịch)</button>
```

```
<button id="button_Truc2_OFF" class="button SYSTEM">DỪNG</button>
</div>

<div style="display: flex; align-items: center; justify-content: center;
gap: 10px;">
  <span>Trục 3 (Step2):</span>
  <div id="LED3Circle_state" class="circle off"></div>
  <button id="button_Truc3_CW" class="button SYSTEM">CW (Thuận)</button>
  <button id="button_Truc3_CCW" class="button SYSTEM">CCW
(Nghịch)</button>
  <button id="button_Truc3_OFF" class="button SYSTEM">DỪNG</button>
</div>
<script>
  // Hàm cập nhật trạng thái hệ thống
  function updateSystemState(state) {
    document.getElementById('SYSTEMtext_state').textContent = state;
    const circle = document.getElementById('SYSTEMCircle_state');
    circle.classList.remove('on', 'off');
circle.classList.add(state.toLowerCase());
    const statusElement = document.getElementById('sensorStatus');
    if (state === 'ON') {
      statusElement.textContent = 'ĐANG HOẠT ĐỘNG';
statusElement.className = 'status enabled';
    } else {
      statusElement.textContent = 'ĐÃ DỪNG'; statusElement.className =
'status disabled';
    }
  }
  function updateLEDState(ledNumber, state) {
    const circle =
document.getElementById(`LED${ledNumber}Circle_state`);
    if (circle) {
      circle.classList.remove('on', 'off', 'cw', 'ccw');
circle.classList.add(state.toLowerCase());
    }
  }
  function initIoTControls() {
    setInterval(() => { fetch('/SYSTEMonstate').then(response =>
response.text()).then(data => updateSystemState(data)); }, 1000);
    setInterval(() => { fetch('/led1state').then(response =>
response.text()).then(data => updateLEDState(1, data)); }, 1000);
    setInterval(() => { fetch('/led2state').then(response =>
response.text()).then(data => updateLEDState(2, data)); }, 1000);
```

```
        setInterval(() => { fetch('/led3state').then(response =>
response.text()).then(data => updateLEDState(3, data)); }, 1000);
    }
    document.addEventListener('DOMContentLoaded', function() {
        document.getElementById('button_SYSTEM_On').addEventListener('click'
, () => fetch('/SYSTEM_ON'));
        document.getElementById('button_SYSTEM_Off').addEventListener('click'
, () => fetch('/SYSTEM_OFF'));
        document.getElementById('resetSYSTEM').addEventListener('click', ()
=> fetch('/RESET_SYSTEM'));
        document.getElementById('button_Truc1_CW').addEventListener('click',
() => fetch('/TRUC1_CW'));
        document.getElementById('button_Truc1_CCW').addEventListener('click'
, () => fetch('/TRUC1_CCW'));
        document.getElementById('button_Truc1_OFF').addEventListener('click'
, () => fetch('/TRUC1_OFF'));
        document.getElementById('button_Truc2_CW').addEventListener('click',
() => fetch('/TRUC2_CW'));
        document.getElementById('button_Truc2_CCW').addEventListener('click'
, () => fetch('/TRUC2_CCW'));
        document.getElementById('button_Truc2_OFF').addEventListener('click'
, () => fetch('/TRUC2_OFF'));
        document.getElementById('button_Truc3_CW').addEventListener('click',
() => fetch('/TRUC3_CW'));
        document.getElementById('button_Truc3_CCW').addEventListener('click'
, () => fetch('/TRUC3_CCW'));
        document.getElementById('button_Truc3_OFF').addEventListener('click'
, () => fetch('/TRUC3_OFF'));
        initIoTControls();
    });
</script>
</body>
</html>
)rawliteral";
```

```
void setup() {
    Serial.begin(115200);

    // --- CẤU HÌNH PHẦN CỨNG ---
    // Động cơ DC
    pinMode(motorPin1, OUTPUT);
    pinMode(motorPin2, OUTPUT);
```

```
pinMode(encoderPinA, INPUT_PULLUP);
pinMode(encoderPinB, INPUT_PULLUP);
// Cấu hình PWM cho động cơ DC trên ESP32
//ledcSetup(0, 5000, 8); // Kênh 0, tần số 5kHz, độ phân giải 8 bit (0-
255)
//ledcAttachPin(enB, 0); // Gán chân enB vào kênh 0
pinMode(enB, OUTPUT);
// Động cơ Bước
pinMode(EnStep1, OUTPUT);
pinMode(EnStep2, OUTPUT);
digitalWrite(EnStep1, HIGH); // Tắt driver
digitalWrite(EnStep2, HIGH); // Tắt driver

// LED báo trạng thái
pinMode(LED1, OUTPUT);
pinMode(LED2, OUTPUT);
pinMode(LED3, OUTPUT);

// Gắn ngắt cho Encoder
attachInterrupt(digitalPinToInterrupt(encoderPinA), updateEncoder,
CHANGE);
attachInterrupt(digitalPinToInterrupt(encoderPinB), updateEncoder,
CHANGE);

// Cấu hình thông số cho AccelStepper
stepper1.setMaxSpeed(1000);
stepper1.setAcceleration(500);
stepper2.setMaxSpeed(1000);
stepper2.setAcceleration(500);

// --- CẤU HÌNH MẠNG VÀ WEB SERVER ---
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("\nWiFi connected.");
Serial.print("IP address: ");
Serial.println(WiFi.localIP());
```

```
server.on("/", handle_MainPage);
server.on("/SYSTEM_ON", handleSystemOn);
server.on("/SYSTEM_OFF", handleSystemOff);
server.on("/RESET_SYSTEM", handleResetSystem);
server.on("/TRUC1_CW", handleTruc1CW);
server.on("/TRUC1_CCW", handleTruc1CCW);
server.on("/TRUC1_OFF", handleTruc1Off);
server.on("/TRUC2_CW", handleTruc2CW);
server.on("/TRUC2_CCW", handleTruc2CCW);
server.on("/TRUC2_OFF", handleTruc2Off);
server.on("/TRUC3_CW", handleTruc3CW);
server.on("/TRUC3_CCW", handleTruc3CCW);
server.on("/TRUC3_OFF", handleTruc3Off);

// Routes lấy trạng thái
server.on("/SYSTEMonstate", handleSystemState);
server.on("/led1state", []() { server.send(200, "text/plain", LED1_State);
});
server.on("/led2state", []() { server.send(200, "text/plain", LED2_State);
});
server.on("/led3state", []() { server.send(200, "text/plain", LED3_State);
});

server.begin();
Serial.println("HTTP server started");

// Đảm bảo tất cả động cơ đều dừng ban đầu
handleSystemOff();
}

void loop() {
server.handleClient(); // Xử lý các yêu cầu web

// Luôn chạy hàm runSpeed() để động cơ bước có thể di chuyển
if(SYSTEM_State == "ON"){
    stepper1.runSpeed();
    stepper2.runSpeed();
}
}

// =====
```

```
// PHẦN 3: TRIỂN KHAI CÁC HÀM XỬ LÝ (HANDLE...) ĐÃ ĐƯỢC CẬP NHẬT
// =====
void stopAllMotors(){
    // Dừng DC
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, LOW);
    ledcWrite(0, 0); // Tốc độ 0
    digitalWrite(LED1, LOW);
    LED1_State = "OFF";

    // Dừng Step1
    stepper1.stop();
    stepper1.setSpeed(0);
    digitalWrite(EnStep1, HIGH); // Tắt driver
    digitalWrite(LED2, LOW);
    LED2_State = "OFF";

    // Dừng Step2
    stepper2.stop();
    stepper2.setSpeed(0);
    digitalWrite(EnStep2, HIGH); // Tắt driver
    digitalWrite(LED3, LOW);
    LED3_State = "OFF";
}

void handleSystemOn() {
    SYSTEM_State = "ON";
    server.send(200, "text/plain", "OK");
    Serial.println("System turned ON");
}

void handleSystemOff() {
    SYSTEM_State = "OFF";
    stopAllMotors();
    server.send(200, "text/plain", "OK");
    Serial.println("System turned OFF");
}

void handleResetSystem() {
    handleSystemOff();
    EncoderCount = 0; // Reset cả bộ đếm encoder
    Serial.println("System RESET");
}
```

```
}

// === TRỤC 1 - ĐỘNG CƠ DC ===
void handleTruc1CW() {
    if (SYSTEM_State == "ON") {
        LED1_State = "CW";
        digitalWrite(motorPin1, HIGH);
        digitalWrite(motorPin2, LOW);
        ledcWrite(0, MANUAL_DC_SPEED);
        digitalWrite(LED1, HIGH);
        server.send(200, "text/plain", "OK");
        Serial.println("Truc 1 (DC) quay thuan (CW)");
    } else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc1CCW() {
    if (SYSTEM_State == "ON") {
        LED1_State = "CCW";
        digitalWrite(motorPin1, LOW);
        digitalWrite(motorPin2, HIGH);
        ledcWrite(0, MANUAL_DC_SPEED);
        digitalWrite(LED1, HIGH);
        server.send(200, "text/plain", "OK");
        Serial.println("Truc 1 (DC) quay nghich (CCW)");
    } else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc1Off() {
    LED1_State = "OFF";
    digitalWrite(motorPin1, LOW);
    digitalWrite(motorPin2, LOW);
    ledcWrite(0, 0);
    digitalWrite(LED1, LOW);
    server.send(200, "text/plain", "OK");
    Serial.println("Truc 1 (DC) dung");
}

// === TRỤC 2 - ĐỘNG CƠ BƯỚC 1 ===
void handleTruc2CW() {
    if (SYSTEM_State == "ON") {
        LED2_State = "CW";
        digitalWrite(EnStep1, LOW); // Bật driver
        stepper1.setSpeed(MANUAL_STEPPER_SPEED);
```



```
        digitalWrite(LED2, HIGH);
        server.send(200, "text/plain", "OK");
        Serial.println("Truc 2 (Step1) quay thuan (CW)");
    } else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc2CCW() {
    if (SYSTEM_State == "ON") {
        LED2_State = "CCW";
        digitalWrite(EnStep1, LOW); // Bật driver
        stepper1.setSpeed(-MANUAL_STEPPER_SPEED);
        digitalWrite(LED2, HIGH);
        server.send(200, "text/plain", "OK");
        Serial.println("Truc 2 (Step1) quay nghich (CCW)");
    } else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc2Off() {
    LED2_State = "OFF";
    stepper1.stop(); // Dừng ngay lập tức
    stepper1.setSpeed(0);
    digitalWrite(EnStep1, HIGH); // Tắt driver để tiết kiệm năng lượng và giảm
    nhiệt
    digitalWrite(LED2, LOW);
    server.send(200, "text/plain", "OK");
    Serial.println("Truc 2 (Step1) dung");
}

// === TRUC 3 - ĐỘNG CƠ BƯỚC 2 ===
void handleTruc3CW() {
    if (SYSTEM_State == "ON") {
        LED3_State = "CW";
        digitalWrite(EnStep2, LOW); // Bật driver
        stepper2.setSpeed(MANUAL_STEPPER_SPEED);
        digitalWrite(LED3, HIGH);
        server.send(200, "text/plain", "OK");
        Serial.println("Truc 3 (Step2) quay thuan (CW)");
    } else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc3CCW() {
    if (SYSTEM_State == "ON") {
```

```
    LED3_State = "CCW";
    digitalWrite(EnStep2, LOW); // Bật driver
    stepper2.setSpeed(-MANUAL_STEPPER_SPEED);
    digitalWrite(LED3, HIGH);
    server.send(200, "text/plain", "OK");
    Serial.println("Truc 3 (Step2) quay nghịch (CCW)");
} else { server.send(200, "text/plain", "SYSTEM_OFF"); }
}

void handleTruc3Off() {
    LED3_State = "OFF";
    stepper2.stop();
    stepper2.setSpeed(0);
    digitalWrite(EnStep2, HIGH); // Tắt driver
    digitalWrite(LED3, LOW);
    server.send(200, "text/plain", "OK");
    Serial.println("Truc 3 (Step2) dung");
}

void handleSystemState() {
    server.send(200, "text/plain", SYSTEM_State);
}

void handle_MainPage() {
    server.send(200, "text/html", htmlPage);
}
```

Điều khiển cánh tay robot bằng ESP32

Điều khiển bằng tay (Jogging Mode)

Trạng thái hệ thống: **ĐÃ DỪNG**

OFF ☐

HOẠT ĐỘNG

DỪNG

RESET

Trục 1 (DC): ☐

CW (Thuận)

CCW (Nghịch)

DỪNG

Trục 2 (Step1): ☐

CW (Thuận)

CCW (Nghịch)

DỪNG

Trục 3 (Step2): ☐

CW (Thuận)

CCW (Nghịch)

DỪNG

Chương 5. KẾT LUẬN

5.1.1. Kết luận

Ứng dụng Arduino ESP32 làm bộ điều khiển trung tâm, lập trình bằng Arduino IDE để điều khiển các động cơ servo.

Điều khiển tự động (chu trình gấp – di chuyển – thả).

Hoàn thiện và thử nghiệm mô hình: robot hoạt động ổn định, có thể gấp được các vật nhỏ (bóng nhựa, khối gỗ nhẹ).

- Trong tương lai, đề tài có thể được mở rộng theo các hướng sau:
- Nâng cấp cơ cấu chấp hành (dùng động cơ bước hoặc servo công suất lớn hơn) để tăng tải trọng.
- Thiết kế lại cấu trúc cơ khí bằng nhôm định hình để tăng độ cứng vững.
- Tích hợp cảm biến và camera để robot có khả năng nhận diện và tự động gấp sản phẩm.
- Kết nối IoT để điều khiển robot từ xa qua Internet.
- Như vậy, thực hiện củng cố kiến thức về robot học, điều khiển nhúng và thiết kế cơ khí – điện tử, mà còn mở ra tiềm năng ứng dụng, nghiên cứu và mô phỏng sản xuất tự động.

5.1.2. Hướng phát triển

Tối ưu thiết kế phần cứng của các thiết bị giúp hoạt động được ổn định sau thời gian dài sử dụng.

Thiết kế phần cứng cánh tay robot nhiều bậc tự do hơn để mở rộng không gian làm việc và có thể sắp xếp hàng hóa theo mong muốn.

Phát triển tính năng bảo mật và vận hành trực tiếp các robot trên web cũng như tối ưu bố cục trình bày.

Tên đề tài: Thiết kế cánh tay Robot gấp sản phẩm sử dụng Arduino

Kết hợp sử dụng các giao thức truyền thông khác để phục vụ cho việc gửi và nhận dữ liệu lớn.

TÀI LIỆU THAM KHẢO

- [1] N. T. Đông, "Chuyên đề internet of things," in *Chuyên đề internet of things*, TP. HCM, TDMU, 2025, p. 41.
- [2] N. T. Đông, "Tựa bài báo," *TDMU*, vol. V1, pp. 52-56, 2025.